

Seccion 1 - Bases de Datos NoSQL y Relacionales

Si bien las bases de datos NoSQL tienen diferencias fundamentales con los sistemas de bases de datos relacionales (RDBMS), algunos conceptos comunes pueden relacionarse. Responder las siguientes preguntas considerando MongoDB en particular como base de datos NoSQL.

1. ¿Cuales de los siguientes conceptos de RDBMS existen en MongoDB? En caso de no existir, ¿hay alguna alternativa? ¿Cual es?
 - Base de Datos
 - Tabla / Relacion
 - Fila / Tupla
 - Columna

En MongoDB existen conceptos equivalentes a varios de los de un RDBMS

RDBMS	MongoDB
Base de Datos	Base de Datos
Tabla / Relación	Colección
Fila / Tupla	Documento
Columna	Campo

2. ¿Existen claves foraneas en MongoDB? ¿Que diferencias existen con las bases de datos de tipo relacional?

No existen claves foráneas como en las DB relacionales, en MongoDB las relaciones normalmente se manejan de dos formas:

- Referencias: guardar el `_id` de otro documento
 - Documentos embebidos: Guardar directamente el documento relacionado dentro del otro
- No hay mecanismos automáticos que garanticen la integridad referencial entre colecciones. La diferencia principal es que en MongoDB la integridad referencial suele quedar a cargo de la aplicación, mientras que en un RDBMS la garantiza el motor de base de datos.

3. Para acelerar las consultas, MongoDB tiene soporte para índices. ¿Que tipos de índices soporta?

Single Field Index

- Índice sobre un único campo
- Compound Index
- Índice sobre múltiples campos
- Geospatial Index

- Para consulta geográficas y coordenadas
Text Index
- Para búsquedas de texto
Hashed Index
- Utilizado principalmente para sharding
Multikey Index
- Se crea automáticamente sobre arrays

4. En MongoDB existen dos tipos de vistas. Explicar brevemente cuales son y que diferencias existen entre ellas. Además, mencionar algunos casos donde podría utilizarlas.

Vistas **estándar**

- Son vistas virtuales
- No almacenan datos físicamente
- Se definen a partir de una consulta o pipeline de aggregation
- Cada vez que se consultan, MongoDB ejecuta nuevamente la consulta sobre las colecciones originales
- Son de sólo lectura
- No pueden tener índices propios

Ventajas

- Los datos siempre están actualizados
 - No ocupan almacenamiento extra
- ##### Desventajas
- Las consultas pueden ser más lentas si el pipeline es complejo

Casos de uso:

- Mostrar recorridos filtrados por precio
- Ocultar ciertos campos sensibles
- Simplificar consultas complejas frecuentes

Vistas **materializadas**

- Almacenan físicamente el resultado
- Generalmente se crean usando \$out o \$merge en aggregation
- Deben actualizarse manualmente o regenerarse periódicamente

Ventajas

- Consultas mucho más rápidas
- Útiles para reportes y estadísticas

Desventajas

- Los datos pueden quedar desactualizados

- Requieren espacio adicional

Casos de uso:

- Reportes agregados
- Estadísticas precalculadas
- Rankings o dashboards

5. Los documentos de una colección pueden diferir en la cantidad y tipos de campos.

¿Existen algunas formas de validar los elementos a insertar en una colección para evitar esta disparidad?

Se pueden utilizar validaciones mediante JSON Schema al crear o modificar una colección
Esto permite

- Definir campos obligatorios
- Restringir tipos de datos
- Validar rangos, tamaños o formatos
- Evitar documentos con estructuras inválidas

6. MongoDB tiene soporte para transacciones, pero no es igual que el de los RDBMS.

¿Cuál es el alcance de una transacción en MongoDB?

Originalmente, MongoDB garantizaba atomicidad únicamente a nivel de documento, pero actualmente soporta transacciones multi-documento y multi-colección

En un RDBMS las transacciones son centrales y se usan constantemente entre múltiples tablas

En MongoDB se prioriza el modelo de agregaciones/documentos embebidos para mantener la atomicidad dentro de un solo documento

7. Las relaciones entre documentos en MongoDB pueden establecerse mediante documentos embebidos o referencias. Investigar cómo se implementa cada una y analizar las ventajas y desventajas de cada una, comparándola con la forma estándar de establecer relaciones en una base de datos relacional.

Documentos embebidos

Consiste en almacenar directamente un documento dentro de otro

Optimizan lecturas y atomicidad

Ventajas

- Lecturas más rápidas
- Toda la información relevante se obtiene en una sola consulta
- Atomicidad garantizada sobre el documento completo
- Evita joins

Desventajas

- Duplicación de información
- Actualizaciones más difíciles si el dato se repite
- El documento puede crecer demasiado

- Limitación de 16MB por documento
Cuando conviene
- Relaciones 1 a pocos
- Datos que casi siempre se consultan juntos
- Datos poco reutilizados

Referencias

Consiste en guardar el `_id` de otro documento

Optimizan reutilización y escalabilidad

Ventajas

- Menor duplicación
- Mejor reutilización de datos
- Documentos más pequeños
- Más flexible para relaciones grandes

Desventajas

- Requiere múltiples consultas o \$lookup
- Menor rendimiento en lecturas complejas
- No existe integridad referencial automática

Cuando conviene

- Relaciones muchos a muchos
- Datos compartidos entre muchos documentos
- Relaciones grandes o dinámicas

Comparación con RDBMS

- En una base relacional:
 - Las relaciones se manejan mediante FK y JOINS
 - La integridad referencial la garantiza el motor
 - El modelo está altamente normalizado
 - En MongoDB
 - Se prioriza el acceso rápido a agregaciones de datos
 - Muchas relaciones se resuelven embebiendo información
 - Las referencias existen, pero la integridad suele manejarse desde aplicación
8. Tomando como referencia el modelo de los trabajos practicos anteriores y suponiendo que este podria mapearse a una base de datos en MongoDB, proponer algunos casos donde la relacion seria conveniente mapearla como referencia y otros como documentos embebidos. Justificar la eleccion.

Referencia

- Las compras de un usuario
 - Un usuario puede tener muchas compras
 - Si fueran embebidas, el documento podría crecer mucho
 - Los datos del usuario se reutilizan constantemente
- Usuarios asociados a recorridos
 - Un mismo usuario puede participar en muchos recorridos
 - Evita duplicar información
- Proveedores y servicios
 - Los proveedores suelen compartirse entre múltiples servicios
 - Son entidades independientes

Documentos embebidos

- Paradas dentro de un recorrido
 - Dado que normalmente las paradas se consultan junto al recorrido
 - Permite obtener toda la información en una sola lectura
 - La cantidad de paradas suele ser pequeña
- Reseñas dentro de una compra
 - Una reseña pertenece únicamente a una compra
 - Se accede normalmente a las reseñas junto a la compra

Seccion 2 - Operaciones CRUD basicas

Descargar la ultima version de MongoDB desde el sitio oficial e ingresar al cliente de linea de comando para realizar los siguientes ejercicios.

2.1 Configuracion inicial e inserciones

9. Crear una nueva base de datos llamada "tours" y una coleccion llamada "recorridos".

No pienso descargar MongoDB local. Abajo hago un contenedor con mongo

```
neftalito@DESKTOP-HP0127H:~$ docker run -d \
  --name mongodb-tours \
  -p 27017:27017 \
  -v mongodb_tours_data:/data/db \
  mongo:latest
```

Ahora uso la shell de mongo

```
neftalito@DESKTOP-HP6127H:~$ docker exec -it mongodb-tours mongosh
Current Mongosh Log ID: 6a05f2837af3a471a49df8a2
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:  8.2.9
Using Mongosh:  2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-05-14T16:03:59.728+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-05-14T16:03:59.874+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-05-14T16:03:59.875+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-05-14T16:03:59.875+00:00: We suggest setting the contents of sysfsFile to 0.
2026-05-14T16:03:59.875+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----
test>
```

Creo la db (use la selecciona y la crea cuando se inserte el primer documento)

```
test> use tours
switched to db tours
```

Creo la colección

```
test> use tours
switched to db tours
tours> db.createCollection("recorridos")
{ ok: 1 }
```

- En la nueva coleccion, utilizando el comando correspondiente, insertar el siguiente documento:

```
{ "nombre": "City Tour", "precio": 200, "stops": ["Diagonal Norte", "Avenida de Mayo", "Plaza del Congreso"], "totalKm": 5 }
```

```
tours> db.recorridos.insertOne({
|   "nombre": "City Tour",
|   "precio": 200,
|   "stops": ["Diagonal Norte", "Avenida de Mayo", "Plaza del Congreso"],
|   "totalKm": 5
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a05f2f17af3a471a49df8a3')
}
```

- Recuperar la informacion insertada usando db.recorridos.find() (puede agregarse .pretty() al final para ver los datos indentados). ¿Que diferencia se observa entre el documento insertado y el documento recuperado?

```
tours> db.recorridos.find().pretty()
[
  {
    _id: ObjectId('6a05f2f17af3a471a49df8a3'),
    nombre: 'City Tour',
    precio: 200,
    stops: [ 'Diagonal Norte', 'Avenida de Mayo', 'Plaza del Congreso' ],
    totalKm: 5
  }
]
```

La diferencia es que el documento insertado no tiene un `_id` y el documento recuperado lo tiene

12. Agregar a la coleccion, utilizando un solo comando, los documentos especificados en el archivo "material_adicional_1.json" adjunto a esta practica.

Como soy un culiado, tengo q copiar el json al docker e importar

```
neftalito@DESKTOP-HP0127H:~$ docker exec -it mongodb-tours mongoimport \
--db tours \
--collection recorridos \
--file /material_adicional_1.json \
--jsonArray
2026-05-14T16:14:19.880+0000    connected to: mongodb://localhost/
2026-05-14T16:14:19.884+0000    14 document(s) imported successfully. 0 document(s) failed to import.
```

2.2 Operaciones de actualizacion y eliminacion

Sobre la coleccion cargada en el punto anterior, realizar las siguientes operaciones:

13. Actualizar el recorrido "Cultural Odyssey" para que su total de kilometros sea 12.

```
tours> db.recorridos.updateOne(
| { nombre: "Cultural Odyssey" },
| { $set: { totalKm: 12 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

14. Actualizar el listado de stops del recorrido "Delta Tour" para agregar "Tigre".

```
tours> db.recorridos.updateOne(
| { nombre: "Delta Tour" },
| { $push: { stops: "Tigre" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

15. Aumentar un 10% el precio de todos los recorridos.

```
tours> db.recorridos.updateMany( {}, { $mul: { precio: 1.10 } } )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 15,
  modifiedCount: 15,
  upsertedCount: 0
}
```

16. Eliminar el recorrido con nombre "Temporal Route".

```
tours> db.recorridos.deleteOne(  
  | { nombre: "Temporal Route" }  
  | )  
{ acknowledged: true, deletedCount: 1 }  
tours>
```

17. Crear el array de etiquetas (tags) para la ruta "Urban Exploration" y agregar el elemento "Gastronomia" a dicho arreglo.

```
tours> db.recorridos.updateOne( { nombre: "Urban Exploration" }, { $push: { tags: "Gastronomia" } })  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

2.3 Consultas con find()

Sobre la coleccion generada anteriormente, realizar las siguientes consultas utilizando la funcion find():

18. Obtener la ruta con nombre "Museum Tour".

```
tours> db.recorridos.find( { nombre: "Museum Tour" } )  
[  
  {  
    _id: ObjectId('6a05f4dbb66aab3cb0c540a7'),  
    nombre: 'Museum Tour',  
    precio: 60500.000000000001,  
    stops: [  
      'Museo Nacional de Bellas Artes',  
      'Teatro Colón',  
      'Planetario',  
      'Bosques de Palermo',  
      'San Telmo',  
      'La Boca - Caminito',  
      'Recoleta',  
      'El Monumental (Estadio River Plate)',  
      'Av 9 de Julio'  
    ],  
    totalKm: 13  
  }  
]
```

19. Las rutas con precio superior a \$60.000.


```

tours> db.recorridos.find({precio: { $gt: 60000 } })
[
  {
    _id: ObjectId('6a05f4dbb66aab3cb0c540a6'),
    nombre: 'Gastronomic Delight',
    precio: 77000,
    stops: [
      'San Telmo',
      'La Boca - Caminito',
      'Belgrano - Barrio Chino',
      'Recoleta',
      'Costanera Sur'
    ],
    totalKm: 9
  },
  {
    _id: ObjectId('6a05f4dbb66aab3cb0c540a7'),
    nombre: 'Museum Tour',
    precio: 60500.000000000001,
    stops: [
      'Museo Nacional de Bellas Artes',
      'Teatro Colón',
      'Planetario',
      'Bosques de Palermo',
      'San Telmo',
      'La Boca - Caminito',
      'Recoleta',
      'El Monumental (Estadio River Plate)',
      'Av 9 de Julio'
    ],
    totalKm: 13
  },
  {
    _id: ObjectId('6a05f4dbb66aab3cb0c540a9'),
    nombre: 'Artistic Journey',
    precio: 66000,
  }
]

```

20. Las rutas con precio superior a \$50.000 y con un total de kilometros mayor a 10.


```

tours> db.recorridos.find({stops: "San Telmo"})
[
  {
    _id: ObjectId('6a05f4dbb66aab3cb0c540a4'),
    nombre: 'Urban Exploration',
    precio: 49500.000000000001,
    stops: [
      'Avenida de Mayo',
      'Museo Moderno',
      'Paseo de la Historieta',
      'Usina del Arte',
      'San Telmo',
      'La Boca - Caminito',
      'Belgrano - Barrio Chino',
      'Recoleta',
      'El Monumental (Estadio River Plate)',
      'Costanera Sur',
      'Plaza Italia'
    ],
    totalKm: 14,
    tags: [ 'Gastronomia' ]
  },
  {
    _id: ObjectId('6a05f4dbb66aab3cb0c540a5'),
    nombre: 'Tango Experience',
    precio: 38500,
    stops: [
      'Avenida de Mayo',
      'Plaza del Congreso',
      'Paseo de la Historieta',
      'San Telmo',
      'La Boca - Caminito',
      'Recoleta'
    ],
    totalKm: 10
  },
  {
    _id: ObjectId('6a05f4dbb66aab3cb0c540a6')
  }
]

```

22. Las rutas que incluyan el stop "Recoleta" y no el stop "Plaza Italia".

```
tours> db.recorridos.find({
|   $and: [
|     { stops: "Recoleta" },
|     { stops: { $ne: "Plaza Italia" } }
|   ]
| })
[
| {
|   _id: ObjectId('6a05f4dbb66aab3cb0c540a5'),
|   nombre: 'Tango Experience',
|   precio: 38500,
|   stops: [
|     'Avenida de Mayo',
|     'Plaza del Congreso',
|     'Paseo de la Historieta',
|     'San Telmo',
|     'La Boca - Caminito',
|     'Recoleta'
|   ],
|   totalKm: 10
| },
| {
|   _id: ObjectId('6a05f4dbb66aab3cb0c540a6'),
|   nombre: 'Gastronomic Delight',
|   precio: 77000,
|   stops: [
|     'San Telmo',
|     'La Boca - Caminito',
|     'Belgrano - Barrio Chino',
|     'Recoleta',
|     'Costanera Sur'
|   ],
|   totalKm: 9
| }
| ]
```

23. El nombre y el total de km (si es que posee) de las rutas que incluyan el stop "Delta" y tengan un precio menor a \$50.000.

```
tours> db.recorridos.find(
| {
|   stops: "Delta",
|   precio: { $lt: 50000 }
| },
| {
|   _id: 0,
|   nombre: 1,
|   totalKm: 1
| }
| )
[ { nombre: 'Nature Escape' } ]
```

24. Las rutas que incluyen tanto "San Telmo" como "Recoleta" y "Avenida de Mayo" entre sus stops.

```

tours> db.recorridos.find({
|   stops: { $all: ["San Telmo", "Recoleta", "Avenida de Mayo"] }
| })
[
  {
    _id: ObjectId('6a05f4dbb66aab3cb0c540a4'),
    nombre: 'Urban Exploration',
    precio: 49500.000000000001,
    stops: [
      'Avenida de Mayo',
      'Museo Moderno',
      'Paseo de la Historieta',
      'Usina del Arte',
      'San Telmo',
      'La Boca - Caminito',
      'Belgrano - Barrio Chino',
      'Recoleta',
      'El Monumental (Estadio River Plate)',
      'Costanera Sur',
      'Plaza Italia'
    ],
    totalKm: 14,
    tags: [ 'Gastronomia' ]
  },
  {
    _id: ObjectId('6a05f4dbb66aab3cb0c540a5'),
    nombre: 'Tango Experience',
    precio: 38500,
    stops: [
      'Avenida de Mayo',
      'Plaza del Congreso',
      'Paseo de la Historieta',
      'San Telmo',
      'La Boca - Caminito',
      'Recoleta'
    ],
    totalKm: 10
  },
  {
    _id: ObjectId('6a05f4dbb66aab3cb0c540ad'),
    nombre: 'Historic Landmarks',
    precio: 27500.000000000004,
    stops: [
      'Avenida de Mayo',
      'Plaza del Congreso',
      'Usina del Arte',
      'San Telmo',
      'La Boca - Caminito',

```

25. Solo el nombre de las rutas que dispongan de mas de 5 stops.

```
tours> db.recorridos.find(
|   { $expr: { $gt: [{ $size: "$stops" }, 5] } },
|   { _id: 0, nombre: 1 }
| )
[
  { nombre: 'Historical Adventure' },
  { nombre: 'City Lights' },
  { nombre: 'Urban Exploration' },
  { nombre: 'Tango Experience' },
  { nombre: 'Museum Tour' },
  { nombre: 'Architectural Expedition' },
  { nombre: 'Artistic Journey' },
  { nombre: 'Historic Landmarks' },
  { nombre: 'Cultural Odyssey' }
]
```

26. Las rutas que no tengan definido el total de sus kilometros.

```
tours> db.recorridos.find({
|   totalKm: { $exists: false }
| })
[
  {
    _id: ObjectId('6a05f4dbb66aab3cb0c540a1'),
    nombre: 'Nature Escape',
    precio: 44000,
    stops: [ 'Delta', 'Río de la Plata', 'Av 9 de Julio', 'Puerto Madero' ]
  }
]
tours>
```

27. Los nombres y el listado de stops de aquellas rutas que incluyen algun museo en sus recorridos.

```

tours> db.recorridos.find(
|   { stops: /museo/i },
|   {
|     nombre: 1,
|     stops: 1,
|     _id: 0
|   }
| )
[
  {
    nombre: 'City Lights',
    stops: [
      'Avenida de Mayo',
      'Paseo de la Historieta',
      'Usina del Arte',
      'Recoleta',
      'El Monumental (Estadio River Plate)',
      'Costanera Sur',
      'Plaza Italia',
      'Museo de Arte Latinoamericano'
    ]
  },
  {
    nombre: 'Urban Exploration',
    stops: [
      'Avenida de Mayo',
      'Museo Moderno',
      'Paseo de la Historieta',
      'Usina del Arte',
      'San Telmo',
      'La Boca - Caminito',
      'Belgrano - Barrio Chino',
      'Recoleta',
      'El Monumental (Estadio River Plate)',
      'Costanera Sur',
      'Plaza Italia'
    ]
  },
  {
    nombre: 'Museum Tour',
    stops: [
      'Museo Nacional de Bellas Artes',
      'Teatro Colón',
      'Planetario',
      'Bosques de Palermo',
      'San Telmo',
      'La Boca - Caminito',

```

También se puede especificar el regex explícitamente

```
db.recorridos.find(
  { stops: { $regex: /museo/i } },
  { _id: 0, nombre: 1, stops: 1 }
)
```

28. La cantidad total de elementos que posee la coleccion.

```
tours> db.recorridos.countDocuments()
14
```

Seccion 3 - Aggregation Framework

Aggregation Framework es una herramienta de MongoDB que permite ampliar la posibilidad de uso de su lenguaje de consulta. Posibilita realizar consultas complejas, filtrado, agrupacion y analisis de datos en tiempo real, aplicando una serie de etapas de procesamiento secuencialmente sobre los documentos de una coleccion.

3.1 Configuracion

29. Crear una nueva base de datos llamada "tours2". Guardar el archivo "generador1.js" adjunto a esta practica y ejecutarlo con:

```
load(<ruta del archivo 'generador1.js'>)
```

Si se utiliza un cliente que lo permita (por ejemplo Robo3T o MongoDB Compass), se puede ejecutar directamente en el espacio de consultas. Examinar las colecciones generadas antes de continuar.

```
tours2> load("/generator1.js")
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
true
```

3.2 Consultas con Aggregation Framework

Utilizando Aggregation Framework, realizar las siguientes operaciones:

Atencion Recordar examinar la estructura de las colecciones generadas por `generador1.js` antes de comenzar las consultas. Las rutas y stops en esta base de datos difieren en estructura de la coleccion de la Seccion 2.

30. Obtener una muestra de 5 rutas aleatorias de la coleccion.


```

tours2> db.route.aggregate([
| { $sample: { size: 5 } }
| ])
[
  {
    _id: ObjectId('6a05ff547af3a471a49ea630'),
    name: 'route44310',
    price: 829,
    totalKm: 11.07,
    stops: [ 57, 28, 98, 64 ]
  },
  {
    _id: ObjectId('6a05ff587af3a471a49eb209'),
    name: 'route47343',
    price: 165,
    totalKm: 11.05,
    stops: [ 12, 35, 82, 87 ]
  },
  {
    _id: ObjectId('6a05ff567af3a471a49eab9b'),
    name: 'route45697',
    price: 674,
    totalKm: 19.4,
    stops: [ 95, 29, 99, 107, 49 ]
  },
  {
    _id: ObjectId('6a05ff547af3a471a49ea5f2'),
    name: 'route44248',
    price: 602,
    totalKm: 7.37,
    stops: [ 95, 118, 94 ]
  },
  {
    _id: ObjectId('6a05ff577af3a471a49eb18a'),
    name: 'route47216',
    price: 758,
    totalKm: 1.51,
    stops: [ 95, 9, 71, 77, 47 ]
  }
]

```

31. Extender la consulta anterior para incluir en el resultado toda la información de cada una de las stops. Tener en cuenta que pueden ligarse por su código.

```

tours2> db.route.aggregate([
|   { $sample: { size: 5 } },
|   {
|       $lookup: {
|           from: "stop",
|           localField: "stops",
|           foreignField: "code",
|           as: "stopsInfo"
|       }
|   }
| ])
[
| {
|     _id: ObjectId('6a05ff537af3a471a49e9fa9'),
|     name: 'route42639',
|     price: 159,
|     totalKm: 18.91,
|     stops: [ 81, 110, 49, 42 ],
|     stopsInfo: [
|         {
|             _id: ObjectId('6a05ff327af3a471a49df8ce'),
|             name: 'Centro Cultural Néstor Kirchner',
|             code: 42,
|             desciprion: 'Stop number 42'
|         },
|         {
|             _id: ObjectId('6a05ff327af3a471a49df8d5'),
|             name: 'Cementerio de la Recoleta',
|             code: 49,
|             desciprion: 'Stop number 49'
|         },
|         {
|             _id: ObjectId('6a05ff327af3a471a49df8f5'),
|             name: 'Museo Enrique Larreta',
|             code: 81,
|             desciprion: 'Stop number 81'
|         },
|         {
|             _id: ObjectId('6a05ff327af3a471a49df912'),
|             name: 'Museo de Arte Hispanoamericano Isaac Fernández Blanco',
|             code: 110,
|             desciprion: 'Stop number 110'
|         }
|     ]
| },
| {
|     _id: ObjectId('6a05ff3f7af3a471a49e3012'),
|     name: 'route14072',
|     price: 627,

```

32. Obtener la informacion de las rutas (incluyendo la de sus stops) que tengan un precio mayor o igual a \$90.000.

Como no hay nada mayor o igual a 90.000, pongo el ejemplo con 900 para que se vea que el filtro funciona

```
tours2> db.route.aggregate([
|   { $match: { price: { $gte: 900 } } },
|   {
|     $lookup: {
|       from: "stop",
|       localField: "stops",
|       foreignField: "code",
|       as: "stopsInfo"
|     }
|   }
| ])
[
| {
|   _id: ObjectId('6a05ff327af3a471a49df92c'),
|   name: 'route18',
|   price: 1000,
|   totalKm: 13.8,
|   stops: [ 49, 97 ],
|   stopsInfo: [
|     {
|       _id: ObjectId('6a05ff327af3a471a49df8d5'),
|       name: 'Cementerio de la Recoleta',
|       code: 49,
|       desciprion: 'Stop number 49'
|     },
|     {
|       _id: ObjectId('6a05ff327af3a471a49df905'),
|       name: 'Centro Cultural de la Memoria Haroldo Conti',
|       code: 97,
|       desciprion: 'Stop number 97'
|     }
|   ]
| },
| {
|   _id: ObjectId('6a05ff327af3a471a49df92d'),
|   name: 'route19',
|   price: 967,
|   totalKm: 17.76,
|   stops: [ 94, 86, 74, 27 ],
|   stopsInfo: [
|     {
|       _id: ObjectId('6a05ff327af3a471a49df8bf'),
|       name: 'Caminito',
|       code: 27,
```

33. Obtener la informacion de las rutas que tengan 5 stops o mas.

```

tours2> db.route.aggregate([
|   {
|     $match: {
|       $expr: { $gte: [{ $size: "$stops" }, 5] }
|     }
|   }
| ])
[
| {
|   _id: ObjectId('6a05ff327af3a471a49df91d'),
|   name: 'route3',
|   price: 537,
|   totalKm: 14.91,
|   stops: [ 37, 5, 86, 15, 41 ]
| },
| {
|   _id: ObjectId('6a05ff327af3a471a49df91f'),
|   name: 'route5',
|   price: 505,
|   totalKm: 7.17,
|   stops: [ 113, 10, 27, 96, 52 ]
| },
| {
|   _id: ObjectId('6a05ff327af3a471a49df920'),
|   name: 'route6',
|   price: 673,
|   totalKm: 18.41,
|   stops: [ 37, 53, 60, 7, 30 ]
| },
| {
|   _id: ObjectId('6a05ff327af3a471a49df925'),
|   name: 'route11',
|   price: 250,
|   totalKm: 4.81,
|   stops: [ 112, 36, 10, 9, 108 ]
| },
| {
|   _id: ObjectId('6a05ff327af3a471a49df928'),
|   name: 'route14',
|   price: 317,
|   totalKm: 1.1,
|   stops: [ 110, 25, 81, 64, 106 ]
| },
| {
|   _id: ObjectId('6a05ff327af3a471a49df92a'),
|   name: 'route16',
|   price: 830,
|   totalKm: 7.21,
|   stops: [ 98, 48, 61, 42, 94 ]
| }
]

```

34. Obtener la información de las rutas que tengan incluido en su nombre el string "111".

```

tours2> db.route.aggregate([
|   { $match: { name: /111/ } }
| ])
[
  {
    _id: ObjectId('6a05ff327af3a471a49df989'),
    name: 'route111',
    price: 867,
    totalKm: 19.75,
    stops: [ 66, 26, 22 ]
  },
  {
    _id: ObjectId('6a05ff337af3a471a49dfd70'),
    name: 'route1110',
    price: 939,
    totalKm: 9.76,
    stops: [ 52, 81, 25 ]
  },
  {
    _id: ObjectId('6a05ff337af3a471a49dfd71'),
    name: 'route1111',
    price: 568,
    totalKm: 10.28,
    stops: [ 35, 113, 45 ]
  },
  {
    _id: ObjectId('6a05ff337af3a471a49dfd72'),
    name: 'route1112',
    price: 164,
    totalKm: 6.13,
    stops: [ 111, 37, 112, 1, 105 ]
  },
  {
    _id: ObjectId('6a05ff337af3a471a49dfd73'),
    name: 'route1113',
    price: 784,
    totalKm: 13.41,
    stops: [ 24, 107 ]
  },
  {
    _id: ObjectId('6a05ff337af3a471a49dfd74'),
    name: 'route1114',
    price: 259,
    totalKm: 12.75,

```

35. Obtener solo las stops de la ruta con nombre "Route100".

```

tours2> db.route.aggregate([
|   { $match: { name: "route100" } },
|   {
|     $lookup: {
|       from: "stop",
|       localField: "stops",
|       foreignField: "code",
|       as: "stopsInfo"
|     }
|   },
|   { $project: { _id: 0, stopsInfo: 1 } }
| ])
[
| {
|   stopsInfo: [
|     {
|       _id: ObjectId('6a05ff327af3a471a49df8d4'),
|       name: 'Centro Cultural Recoleta',
|       code: 48,
|       desciprion: 'Stop number 48'
|     },
|     {
|       _id: ObjectId('6a05ff327af3a471a49df8f7'),
|       name: 'Barrancas de Belgrano',
|       code: 83,
|       desciprion: 'Stop number 83'
|     },
|     {
|       _id: ObjectId('6a05ff327af3a471a49df8fa'),
|       name: 'Recoleta',
|       code: 86,
|       desciprion: 'Stop number 86'
|     },
|     {
|       _id: ObjectId('6a05ff327af3a471a49df8fe'),
|       name: 'Museo Nacional de Bellas Artes',
|       code: 90,
|       desciprion: 'Stop number 90'
|     }
|   ]
| }
| ]
tours2>

```

36. Obtener la informacion del stop que mas apariciones tiene en rutas.

```
tours2> db.route.aggregate([
|   { $unwind: "$stops" },
|   { $group: { _id: "$stops", apariciones: { $sum: 1 } } },
|   { $sort: { apariciones: -1 } },
|   { $limit: 1 },
|   {
|     $lookup: {
|       from: "stop",
|       localField: "_id",
|       foreignField: "code",
|       as: "stopInfo"
|     }
|   },
|   { $project: { _id: 0, apariciones: 1, stopInfo: { $arrayElemAt: ["$stopInfo", 0] } } }
| ])
|
| {
|   apariciones: 1568,
|   stopInfo: {
|     _id: ObjectId('6a05ff327af3a471a49df908'),
|     name: 'Paseo de la Gloria',
|     code: 100,
|     desciprion: 'Stop number 100'
|   }
| }
| ]
```

37. Obtener las rutas con precio inferior a \$15.000. Agregar a cada una una nueva propiedad que especifique la cantidad de stops que posee. Crear una nueva colección llamada "rutas_economicas" y almacenar estos elementos.

```
tours2> db.route.aggregate([
|   { $match: { price: { $lt: 15000 } } },
|   { $addFields: { stopCount: { $size: "$stops" } } },
|   { $out: "rutas_economicas" }
| ])
```

Como el generador nunca genera precios mayores a 15.000, entonces todas las rutas serán agregadas a esa colección

Vamos a tomar como ejemplo rutas con precio menor a 500 para ver que se agrega correctamente a la nueva colección

Primero se borra la colección anterior

```
tours2> db.rutas_economicas.drop()
true
```

La creamos nuevamente con rutas con precio menor a 500

```
tours2> db.route.aggregate([
|   { $match: { price: { $lt: 500 } } },
|   { $addFields: { stopCount: { $size: "$stops" } } },
|   { $out: "rutas_economicas" }
| ])
```

Estas son algunas de las rutas añadidas

```
tours2> db.rutas_economicas.find()
[
  {
    _id: ObjectId('6a05ff327af3a471a49df91b'),
    name: 'route1',
    price: 181,
    totalKm: 11.36,
    stops: [ 56, 89, 52, 12 ],
    stopCount: 4
  },
  {
    _id: ObjectId('6a05ff327af3a471a49df921'),
    name: 'route7',
    price: 303,
    totalKm: 16.82,
    stops: [ 102, 80, 46, 0 ],
    stopCount: 4
  },
  {
    _id: ObjectId('6a05ff327af3a471a49df925'),
    name: 'route11',
    price: 250,
    totalKm: 4.81,
    stops: [ 112, 36, 10, 9, 108 ],
    stopCount: 5
  },
  {
    _id: ObjectId('6a05ff327af3a471a49df927'),
    name: 'route13',
    price: 299,
    totalKm: 1.94,
    stops: [ 9, 47, 13 ],
    stopCount: 3
  },
  {
    _id: ObjectId('6a05ff327af3a471a49df928'),
```


38. Por cada stop existente en la coleccion, calcular el precio promedio de las rutas que la incluyen.

Tip Si una consulta de aggregation comienza a ser dificil de leer, se pueden guardar las etapas individuales en variables JavaScript antes de armar el pipeline. Cada etapa no es mas que un objeto JSON.

```
tours2> db.route.aggregate([
  { $unwind: "$stops" },
  { $group: { _id: "$stops", precioPromedio: { $avg: "$price" } } },
  {
    $lookup: {
      from: "stop",
      localField: "_id",
      foreignField: "code",
      as: "stopInfo"
    }
  },
  {
    $project: {
      _id: 0,
      stop: { $arrayElemAt: ["$stopInfo.name", 0] },
      precioPromedio: { $round: ["$precioPromedio", 2] }
    }
  },
  { $sort: { stop: 1 } }
])

[
  { stop: 'Arco Inaugural Barrio Chino', precioPromedio: 557.99 },
  { stop: 'Av 9 de Julio', precioPromedio: 552.5 },
  { stop: 'Avenida de Mayo', precioPromedio: 551.04 },
  { stop: 'Barrancas de Belgrano', precioPromedio: 551.18 },
  { stop: 'Basílica Nuestra Señora del Pilar', precioPromedio: 555.14 },
  { stop: 'Basílica Nuestra Señora del Pilar', precioPromedio: 550.86 },
  { stop: 'Basílica de San Francisco', precioPromedio: 553.82 },
  { stop: 'Belgrano - Barrio Chino', precioPromedio: 547.5 },
  { stop: 'Biblioteca Nacional', precioPromedio: 544.8 },
  { stop: 'Bosques de Palermo', precioPromedio: 553.31 },
  { stop: 'Buque Escuela Corbeta Uruguay', precioPromedio: 567.61 },
  { stop: 'Cabildo de Buenos Aires', precioPromedio: 549.96 },
  { stop: 'Cabildo de Buenos Aires', precioPromedio: 555.28 },
  { stop: 'Café Tortoni', precioPromedio: 554.91 },
  { stop: 'Caminito', precioPromedio: 545.99 },
  { stop: 'Campo de Polo', precioPromedio: 555.24 },
  { stop: 'Casa Rosada', precioPromedio: 548.15 },
  { stop: 'Casa Rosada', precioPromedio: 549.28 },
  { stop: 'Catedral Metropolitana', precioPromedio: 537.83 },
  { stop: 'Cementerio de la Recoleta', precioPromedio: 531.82 }
]
```